

Graph RAG - 관계형 검색 원리

1. Graph RAG 구현

1. Graph RAG의 등장 배경

일반적인 RAG(Retrieval-Augmented Generation)는 사용자의 질문과 의미적으로 유사한 문서를 검색한 후 검색된 문서를 LLM에게 제공하여 답변을 생성하는 방식이다.

기본적인 RAG의 처리 과정은 다음과 같다.

사용자 질문



질문 임베딩



Vector DB 유사도 검색



관련 문서 Chunk 검색



LLM에 Context 전달



답변 생성



예를 들어 다음과 같은 질문을 생각할 수 있다.

"삼성전자의 주요 반도체 사업과 관련된 기업들은 무엇이며, 이 기업들과의 관계는 어떻게 되는가?"

일반적인 Vector RAG는 질문과 의미적으로 유사한 문서를 검색하는 데는 강점이 있다.

하지만 다음과 같은 문제가 발생할 수 있다.

- 여러 문서에 분산된 정보를 연결하기 어렵다.
- 사람, 기업, 제품, 기술 등의 관계를 명확하게 표현하기 어렵다.
- 여러 단계의 관계를 따라가야 하는 질문에 약하다.
- "A와 관련된 B의 C"와 같은 다중 홉(Multi-hop) 질문에서 검색 결과가 부족할 수 있다.
- 검색된 Chunk들이 서로 어떤 관계가 있는지 알기 어렵다.

이러한 문제를 해결하기 위한 방법 중 하나가 Graph RAG이다.

2. Graph RAG란?

Graph RAG(Graph-based Retrieval-Augmented Generation)는 문서에서 추출한 개체(Entity)와 관계(Relationship)를 그래프 구조로 표현하고, 사용자의 질문에 관련된 관계를 탐색하여 검색 결과를 구성하는 RAG 방식이다.

핵심적인 차이는 다음과 같다.

일반적인 RAG:

질문



"비슷한 문서" 검색



관련 Chunk



LLM



Graph RAG:

질문



관련 Entity 파악



관련 관계 탐색



연결된 Entity 검색



관련 정보 구성



LLM

즉, Graph RAG는 단순히 "비슷한 내용을 찾는 것"을 넘어

"질문과 관련된 개체들이 서로 어떻게 연결되어 있는가?"

를 검색하는 방식이다.

3. 핵심 개념

Graph RAG를 이해하기 위해서는 먼저 Knowledge Graph의 기본 구조를 이해해야 한다.

지식 그래프는 일반적으로 다음 세 가지 요소로 구성된다.

Entity — Relationship — Entity

예를 들어,

삼성전자 — 생산한다 — DRAM

이를 그래프의 관점에서 보면 다음과 같다.

[삼성전자]
|
생산
|
▼
[DRAM]

여기에서

- 삼성전자 → Entity
- DRAM → Entity
- 생산한다 → Relationship

이다.

이를 Triple이라고 하며 다음과 같이 표현할 수 있다.

(삼성전자, 생산한다, DRAM)

이를 일반적으로

Subject - Predicate - Object

형태로 표현한다.

Subject	Predicate	Object
삼성전자	생산한다	DRAM

이러한 Triple들이 모이면 하나의 지식 그래프가 만들어진다.

4. Knowledge Graph의 기본 구조

예를 들어 다음과 같은 기업 정보를 가지고 있다고 가정한다.

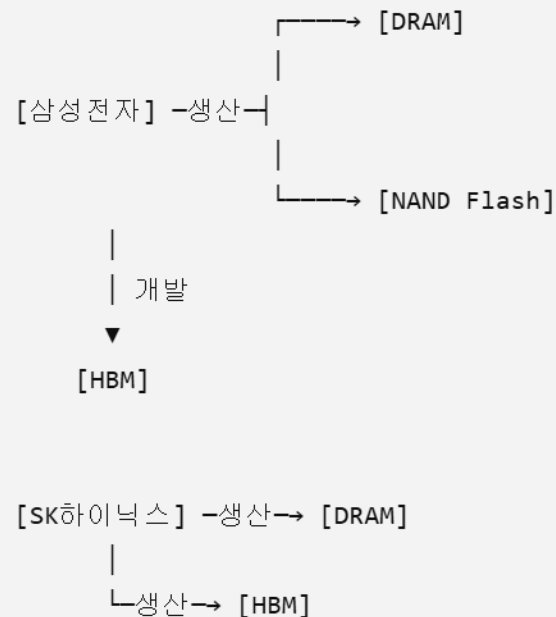
삼성전자

- └─ 생산 → DRAM
- └─ 생산 → NAND Flash
- └─ 개발 → HBM
- └─ 투자 → 반도체 연구개발

SK하이닉스

- └─ 생산 → DRAM
- └─ 생산 → HBM
- └─ 개발 → AI Memory

이를 그래프로 표현하면 다음과 같다.



여기서 중요한 것은 DRAM과 HBM을 통해 삼성전자와 SK하이닉스가 연결될 수 있다는 것이다.

삼성 전자



생산



HBM



생산



SK하이닉스

이러한 "연결 관계"가 Graph RAG의 핵심이다.

5. Vector RAG와 Graph RAG의 차이

5.1 Vector RAG

Vector RAG에서는 문서를 여러 개의 Chunk로 분할한다.

예를 들어 문서가 다음과 같다고 가정한다.

Chunk 1

삼성전자는 HBM3E 제품을 개발하고 있다.

Chunk 2

SK하이닉스는 HBM3E를 주요 제품으로 생산하고 있다.

Chunk 3

HBM은 AI 가속기에 사용되는 고대역폭 메모리이다.

사용자가

"HBM과 관련된 기업은?"

이라고 질문하면 Vector DB에서 질문과 의미적으로 유사한 Chunk를 검색한다.

Query

↓

Embedding

↓

Vector DB

↓

Chunk 1

Chunk 2

Chunk 3

문제는 검색된 Chunk 사이의 관계가 명시적으로 표현되지 않는다는 것이다.

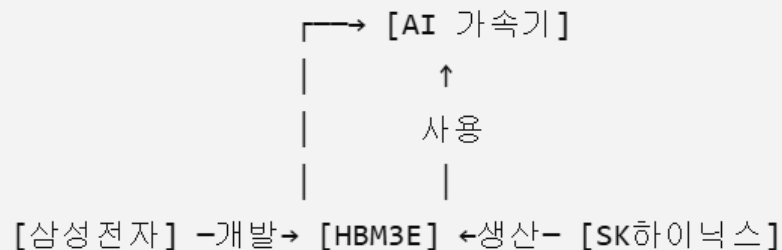
5.2 Graph RAG

Graph RAG에서는 문서에서 Entity와 Relationship을 추출한다.

예를 들어:

삼성전자 → 개발 → HBM3E
SK하이닉스 → 생산 → HBM3E
HBM3E → 사용 → AI 가속기

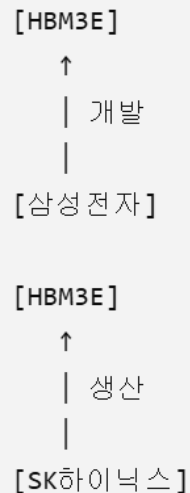
이를 그래프로 구성한다.



이제 질문이

"HBM3E와 관련된 기업은?"

이라면 HBM3E를 중심으로 연결된 Entity를 탐색할 수 있다.



즉, 검색의 중심이 "문서의 유사도"에서 "Entity 간 관계"로 확장된다.

6. Graph RAG에서 사용하는 주요 용어

6.1 Node

Graph의 각각의 객체를 Node라고 한다.

예:

[삼성전자]
[SK하이닉스]
[HBM]
[DRAM]
[AI 가속기]

Node는 일반적으로 다음과 같은 Entity를 표현한다.

- 사람
- 기업
- 제품
- 기술
- 장소
- 기관
- 사건
- 문서
- 개념

6.2 Edge

Node와 Node를 연결하는 관계를 Edge라고 한다.

예:

삼성전자 — 생산 — HBM

여기에서 "생산"이 Edge이다.

다른 예:

홍길동 — 소속 — 삼성전자

삼성전자 — 개발 — HBM

HBM — 사용 — AI 가속기

6.3 Property

Node나 Edge에 추가적인 속성을 저장할 수도 있다.

예를 들어 기업 Node에 다음 정보를 저장할 수 있다.

```
삼성전자
├ type: company
├ country: Korea
├ founded: 1969
└ industry: semiconductor
```

Relationship에도 속성을 추가할 수 있다.

```
삼성전자
├
└ 생산
  ├── product: HBM3E
  └ year: 2025
```

따라서 실제 Knowledge Graph는 단순한 연결선이 아니라 상당히 많은 정보를 포함할 수 있다.

7. Graph RAG의 핵심: 관계형 검색

Graph RAG에서 가장 중요한 개념은 관계형 검색(Relational Search)이다.

Vector Search는 기본적으로 다음과 같은 질문에 답한다.

"이 질문과 의미적으로 비슷한 문서는 무엇인가?"

반면 Graph Search는 다음과 같은 질문에 답한다.

"이 Entity와 연결된 Entity는 무엇인가?"

예를 들어 다음과 같은 그래프가 있다고 하자.

[삼성전자]
|
개발
↓
[HBM3E]
|
사용
↓
[AI 가속기]
|
공급
↓
[NVIDIA]

질문:

"삼성전자와 NVIDIA는 어떤 기술을 통해 연결되는가?"

단순 Vector Search보다 Graph Search가 자연스럽다.

Graph를 따라가면 다음 경로를 찾을 수 있기 때문이다.

삼성전자
↓
HBM3E
↓
AI 가속기
↓
NVIDIA

이것이 관계형 검색의 핵심 원리이다.

8. Graph Traversal

그래프에서 Node 사이의 관계를 따라가는 것을 Graph Traversal이라고 한다.

예를 들어 다음과 같은 그래프가 있다고 하자.

$A \rightarrow B \rightarrow C \rightarrow D$

A에서 출발하여 C를 찾으려면

$A \rightarrow B \rightarrow C$

라는 경로를 탐색한다.

Graph RAG에서는 이러한 탐색을 이용하여 질문과 관련된 정보를 확장한다.

예:

질문

"삼성전자가 개발하는 제품과
그 제품을 사용하는 기업은?"



[삼성전자]



개발



[HBM3E]



사용



[NVIDIA]

따라서 최종 검색 결과는 단순히 삼성전자에 대한 문서가 아니라

삼성전자



HBM3E



NVIDIA

라는 관계 구조를 포함한다.

9. 1-Hop, 2-Hop, Multi-Hop 검색

Graph RAG에서는 몇 단계의 관계를 탐색하는지가 중요하다.

9.1 1-Hop Search

바로 연결된 Node를 검색한다.

[삼성전자]

|

└─ [HBM]

└─ [DRAM]

└─ [NAND]

질문:

"삼성전자가 생산하는 제품은?"

1-Hop 검색으로 해결할 수 있다.

9.2 2-Hop Search

두 단계의 관계를 탐색한다.

[삼성전자]

|

생산

↓

[HBM]

|

사용

↓

[AI 가속기]

질문:

"삼성전자가 생산하는 메모리가 사용되는 제품은?"

2-Hop 탐색이 필요하다.

삼성전자

→ HBM

→ AI 가속기

9.3 Multi-Hop Search

여러 단계의 관계를 연결한다.

삼성전자
↓
HBM
↓
AI 가속기
↓
NVIDIA
↓
데이터센터

질문:

"삼성전자의 HBM 기술은 어떤 기업의 AI 가속기를 통해 데이터센터에 활용되는가?"

이 경우 여러 관계를 따라가야 한다.

이 경우 여러 관계를 따라가야 한다.

삼성전자

- HBM
- AI 가속기
- NVIDIA
- 데이터센터

이러한 질문을 Multi-hop Question이라고 한다.

Graph RAG가 특히 강점을 가지는 영역이다.

10. 왜 일반 Vector RAG만으로 부족한가?

다음과 같은 정보가 여러 문서에 분산되어 있다고 가정한다.

문서 A:

삼성전자는 HBM3E를 개발하였다.

문서 B:

HBM3E는 AI 가속기에 사용된다.

문서 C:

NVIDIA는 AI 가속기를 개발한다.

문서 D:

NVIDIA의 AI 가속기는 데이터센터에서 사용된다.

질문:

"삼성전자의 HBM3E 기술은 어떤 기업의 제품을 통해 데이터센터에 활용되는가?"

정답을 만들려면 다음 정보를 연결해야 한다.

삼성 전자



HBM3E



AI 가속기



NVIDIA



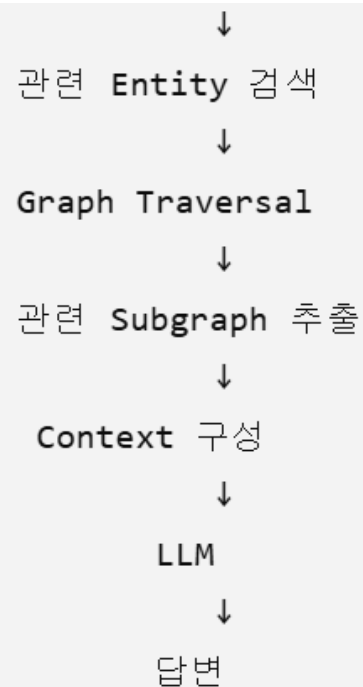
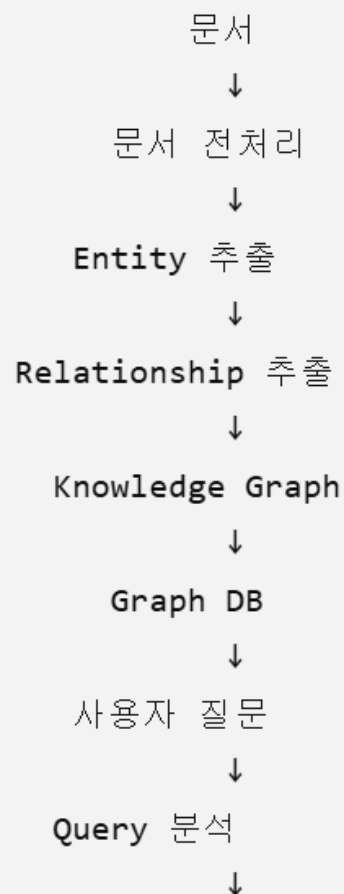
데이터센터

즉, 개별 문서를 검색하는 것뿐만 아니라 여러 문서에 존재하는 관계를 연결해야 한다.

Graph RAG는 이러한 연결 관계를 그래프에 미리 표현할 수 있다.

11. Graph RAG의 전체 처리 과정

Graph RAG의 기본적인 구축 과정은 다음과 같다.



이를 크게 두 단계로 나눌 수 있다.

1. Graph 구축 단계
2. Graph 검색 단계

12. Graph 구축 단계

Step 1. 문서 수집

PDF, 웹 문서, 보고서, 뉴스, 기술 문서 등을 수집한다.

예:

```
documents/  
├─ samsung.pdf  
├─ sk_hynix.pdf  
├─ nvidia.pdf  
└─ semiconductor_report.pdf
```

Step 2. 문서 분할

일반 RAG와 마찬가지로 문서를 적절한 단위로 분할한다.

Document



Chunk 1

Chunk 2

Chunk 3

...



Graph RAG에서도 원문 Chunk를 보존하는 것이 중요하다.

왜냐하면 그래프의 Entity와 Relationship을 추출하더라도 최종 답변을 생성할 때 원문 근거가 필요하기 때문이다.

13. Entity Extraction

LLM을 사용하여 문서에서 중요한 Entity를 추출한다.

예를 들어 문장:

"삼성전자는 2025년 HBM3E 생산을 확대하고 NVIDIA에 공급을 확대하였다."

에서 다음과 같은 Entity를 추출할 수 있다.

삼성전자

HBM3E

NVIDIA

2025년

Entity 유형도 지정할 수 있다.

삼성전자 → Company

HBM3E → Product

NVIDIA → Company

2025년 → Date

14. Relationship Extraction

다음으로 Entity 사이의 관계를 추출한다.

원문:

"삼성전자는 HBM3E를 생산하여 NVIDIA에 공급하였다."

추출 결과:

삼성전자 — 생산 —> HBM3E

삼성전자 — 공급 —> NVIDIA

Triple로 표현하면 다음과 같다.

(삼성전자, 생산, HBM3E)

(삼성전자, 공급, NVIDIA)

15. Knowledge Graph 생성

여러 문서에서 추출한 Entity와 Relationship을 통합한다.

예:

```
[삼성전자]
  |
  | 생산
  | ↓
[HBM3E]
  |
  | 사용
  | ↓
[AI 가속기]
  ↑
  | 개발
  |
[NVIDIA]
```

실제 그래프 DB에서는 수많은 Node와 Edge가 연결될 수 있다.

```
Company
  |
  |— produces → Product
  |
  |— develops → Technology
  |
  |— supplies → Company
  |
  |— partners → Company
```

16. 질문 처리 단계

사용자가 질문을 입력하면 먼저 질문에서 핵심 Entity와 관계를 파악한다.

질문:

"삼성전자가 생산하는 HBM이 사용되는 AI 가속기 관련 기업은?"

핵심 Entity:

삼성전자

HBM

AI 가속기

질문의 의도:

삼성전자

↓

HBM

↓

AI 가속기

↓

기업

따라서 Graph에서 관련 경로를 탐색한다.

17. Subgraph 검색

전체 Knowledge Graph를 LLM에게 전달하는 것은 비효율적이다.

예를 들어 전체 그래프가 수십만 개의 Node를 가지고 있다고 하자.

전체 Knowledge Graph



수십만 Node



질문과 관련된 Subgraph만 추출



LLM

질문과 관련된 부분만 추출하는 것을 Subgraph Retrieval이라고 볼 수 있다.

예:

전체 Graph

A - B - C - D - E

|

F - G - H

|

I

↓

질문과 관련된 Subgraph

B - C - D

|

G

이렇게 검색된 작은 그래프를 LLM의 Context로 전달한다.

18. Graph RAG의 검색 방식

Graph RAG의 검색은 여러 방법으로 구현할 수 있다.

대표적인 방식은 다음과 같다.

1. Entity 기반 검색

질문에서 Entity를 추출한 후 해당 Entity를 검색한다.

```
Query
↓
Entity Extraction
↓
Entity Search
```

2. Neighbor Search

검색된 Entity와 연결된 주변 Node를 검색한다.

```
Entity
↓
1-Hop Neighbor
↓
관련 Node
```

3. Multi-Hop Traversal

여러 단계의 관계를 따라간다.

Entity



1-Hop



2-Hop



3-Hop

4. Path Search

두 Entity 사이의 연결 경로를 검색한다.

A → B → C → D

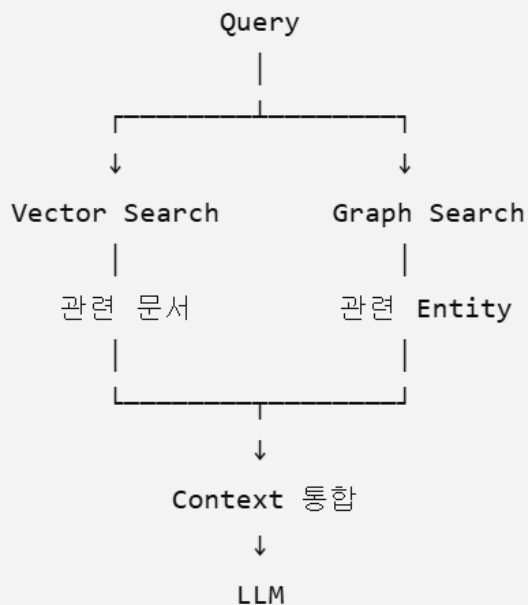
5. Community Search

대규모 그래프를 여러 Community로 나누고 질문과 관련된 Community를 검색한다.

19. Entity Search와 Vector Search의 결합

실제 Graph RAG에서는 Vector Search와 Graph Search를 함께 사용하는 경우가 많다.

이를 Hybrid Graph RAG라고 볼 수 있다.



Vector Search는 "내용의 의미적 유사성"을 찾는 데 강하고,

Graph Search는 "Entity 간 관계"를 찾는 데 강하다.

따라서 두 검색 결과를 결합하면 검색 품질을 높일 수 있다.

20. Vector RAG와 Graph RAG 비교

구분	Vector RAG	Graph RAG
기본 검색	의미적 유사도	관계 및 연결
주요 단위	Chunk	Entity / Relationship
핵심 구조	Vector	Graph
검색 방식	Similarity Search	Graph Traversal
강점	유사 문서 검색	관계 탐색
Multi-hop 질문	상대적으로 어려움	강점
Entity 관계	명시적이지 않음	명시적
구현 난이도	낮음	높음
구축 비용	상대적으로 낮음	상대적으로 높음
업데이트	비교적 단순	Graph 갱신 필요
적합한 질문	"무엇인가?"	"무엇과 어떻게 연결되는가?"

21. Graph RAG가 특히 유용한 질문

다음과 같은 질문은 Graph RAG의 장점을 잘 보여준다.

관계 질문

A와 B는 어떤 관계인가?

A → 관계 → B

연결 질문

A와 C를 연결하는 중간 개체는 무엇인가?

A → B → C

Multi-hop 질문

A가 개발한 기술이 어떤 제품을 거쳐 어떤 기업에서 사용되는가?

A
↓
기술
↓
제품
↓
기업

조직 관계

특정 임원이 소속된 회사와 해당 회사의 주요 자회사는?

임원
↓ 소속
회사
↓ 자회사
자회사

공급망 분석

특정 기업의 제품 생산에 필요한 핵심 공급업체는?

제품
↑
생산
↑
기업
↑
공급
↑
공급업체

22. Graph RAG가 유리한 실제 사례

금융

기업
↓
계열사
↓
자회사
↓
투자기업
↓
산업

질문:

"A 기업의 계열사 중 반도체 산업과 관련된 기업은?"

의료

질병
↓
증상
↓
검사
↓
치료제
↓
제약회사

질문:

"특정 질병과 관련된 치료제와 해당 치료제를 개발한 기업은?"

제조

제품
↓
부품
↓
공급업체
↓
원자재

질문:

"특정 제품의 생산에 영향을 미치는 주요 공급업체는?"

스마트팜

사용자가 진행하는 스마트 농업 분야에서도 Graph RAG를 적용할 수 있다.

예:

작물
↓
질병
↓
증상
↓
원인
↓
방제 방법
↓
농약

질문:

"토마토 잎에 황색 반점이 나타날 때 관련된 병해와 방제 방법은?"

또는

작물



품종



재배환경



온도



생육장애



관리방법



질문:

"토마토의 야간 온도가 낮을 때 발생할 수 있는 생육 문제와 관리 방법은?"

이러한 관계를 Knowledge Graph로 구성하면 단순한 문서 검색보다 관계를 중심으로 정보를 연결할 수 있다.

23. Graph RAG의 대표적인 데이터 구조

가장 기본적인 구조는 Triple이다.

(Subject, Predicate, Object)

예:

(삼성전자, 생산, HBM)

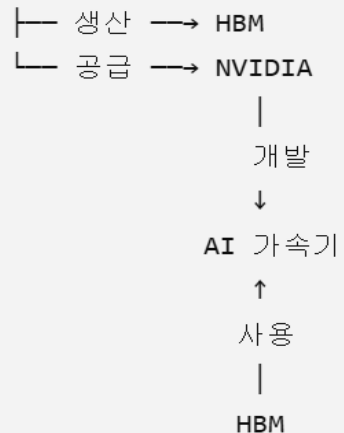
(삼성전자, 공급, NVIDIA)

(HBM, 사용, AI가속기)

(NVIDIA, 개발, AI가속기)

이를 그래프로 연결한다.

삼성전자



이 구조는 Neo4j와 같은 Graph Database에 저장할 수 있다.

24. Neo4j와 Graph RAG

Graph RAG 실습에서는 Neo4j를 대표적인 Graph Database로 사용할 수 있다.

Neo4j는 데이터를 Node와 Relationship 형태로 저장한다.

개념적으로 다음과 같다.

```
(:Company {name:"삼성전자"})  
  |  
  | PRODUCES  
  ↓  
(:Product {name:"HBM3E"})
```

Cypher라는 그래프 질의 언어를 사용하여 관계를 검색할 수 있다.

예를 들어 개념적으로 다음과 같은 검색이 가능하다.

```
<> cypher
```

```
MATCH (c:Company)-[:PRODUCES]->(p:Product)  
WHERE c.name = "삼성전자"  
RETURN c, p
```

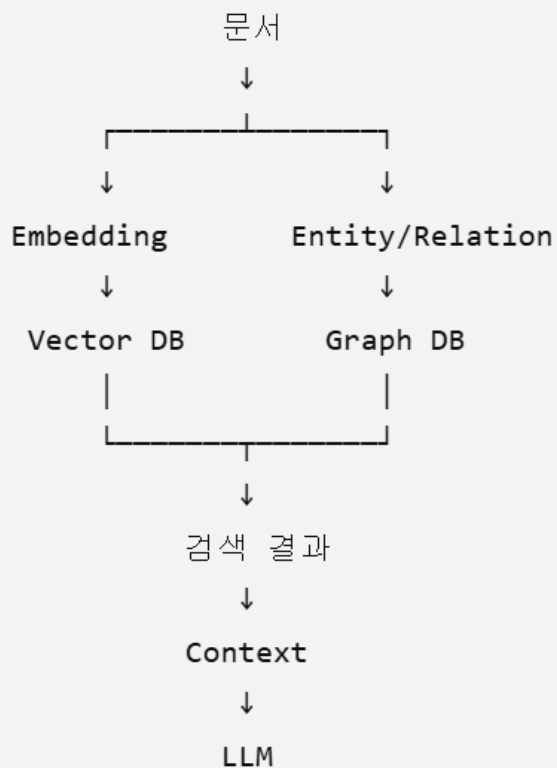
이 쿼리는

```
삼성전자  
  ↓ PRODUCES  
HBM3E
```

와 같은 관계를 검색한다.

25. Vector DB + Graph DB 구조

실제 RAG 시스템에서는 Vector DB와 Graph DB를 함께 사용하는 구조가 효과적이다.



각 DB의 역할이 다르다.

각 DB의 역할이 다르다.

Vector DB:

"질문과 의미적으로 유사한 문서"

Graph DB:

"질문의 Entity와 연결된 정보"

이 두 가지를 결합하는 것이 실전적인 Graph RAG 구조이다.

26. Graph RAG의 장점

1. 관계 정보를 명시적으로 표현

일반 Vector DB에서는 관계가 암묵적으로 존재한다.

Graph에서는 관계가 명시적으로 존재한다.

A — 관계 —> B

2. Multi-hop 검색

여러 단계의 관계를 따라갈 수 있다.

A → B → C → D

3. 설명 가능한 검색

왜 해당 정보가 검색되었는지 경로를 설명할 수 있다.

예:

삼성전자
↓ 생산
HBM3E
↓ 사용
AI 가속기
↓ 개발
NVIDIA

따라서 검색 결과가 단순한 문서 목록이 아니라 "관계 경로"로 설명될 수 있다.

4. 복잡한 관계형 질문에 강함

특히 다음과 같은 분야에서 효과적이다.

- 금융
- 의료
- 제조
- 공급망
- 법률
- 연구 논문
- 기업 분석
- 지식 관리

27. Graph RAG의 한계

Graph RAG가 모든 질문에서 Vector RAG보다 좋은 것은 아니다.

1. Graph 구축 비용

Entity와 Relationship을 추출해야 하므로 구축 과정이 복잡하다.

문서
↓
Entity Extraction
↓
Relationship Extraction
↓
Entity 정규화
↓
Graph 생성

2. Entity 추출 오류

LLM이 잘못된 Entity를 추출할 수 있다.

예:

삼성전자
삼성전자 주식
삼성전자 반도체

이들이 동일한 Entity인지 구분해야 한다.

이를 Entity Resolution 또는 Entity Linking 문제라고 한다.

3. Relationship 추출 오류

원문:

"A사는 B사와 협력 가능성을 검토하고 있다."

이를 잘못 해석하여

A사 — 협력 — B사

로 저장하면 실제 사실과 다른 관계가 만들어질 수 있다.

따라서 Graph의 품질은 원문 분석 품질에 크게 영향을 받는다.

4. Graph 유지보수

새로운 문서가 추가되면 기존 Graph도 업데이트해야 한다.

새 문서



새 Entity



새 Relationship



기존 Graph와 통합

28. Graph RAG에서 중요한 Entity Resolution

실제 Graph 구축에서 매우 중요한 문제가 Entity 중복이다.

예를 들어 문서마다 다음과 같이 표현될 수 있다.

삼성전자
삼성전자(주)
Samsung Electronics
Samsung Electronics Co., Ltd.

사람이 보면 동일한 기업이지만 시스템에서는 서로 다른 문자열이다.

잘못 구축하면 다음처럼 된다.

[삼성전자]
[삼성전자(주)]
[Samsung Electronics]
[Samsung Electronics Co., Ltd.]

실제로는 하나의 Entity여야 한다.

┌ 삼성전자
├ 삼성전자(주)
├ Samsung Electronics
└ Samsung Electronics Co., Ltd.
↓
[삼성전자]

이 과정을 Entity Resolution이라고 한다.

29. Graph RAG에서 중요한 Relationship Normalization

관계도 정규화해야 한다.

예를 들어 문서에 다음 표현이 존재할 수 있다.

생산한다
제조한다
만든다
생산
제조

모두 같은 관계로 판단할 수 있다.

PRODUCES

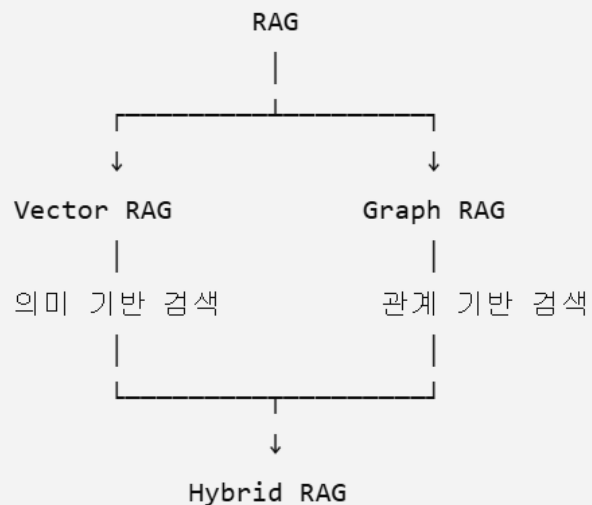
따라서 Graph를 만들 때 관계 유형을 일정한 Vocabulary로 통일할 필요가 있다.

예:

PRODUCES
DEVELOPS
SUPPLIES
OWNS
PARTNERS_WITH
LOCATED_IN
USED_BY

30. Graph RAG와 일반 RAG의 관계

Graph RAG는 Vector RAG를 완전히 대체하는 기술이라기보다 보완하는 기술로 이해하는 것이 좋다.



따라서 실전에서는

Vector Search
+
Graph Search
+
LLM

구조를 많이 고려한다.

31. Graph RAG의 핵심 원리 정리

Graph RAG의 핵심은 다음 네 가지로 정리할 수 있다.

① Entity

"무엇이 존재하는가?"

삼성전자
HBM
NVIDIA

② Relationship

"무엇과 무엇이 어떤 관계인가?"

삼성전자 - 생산 → HBM

③ Traversal

"관계를 따라 어디까지 연결되는가?"

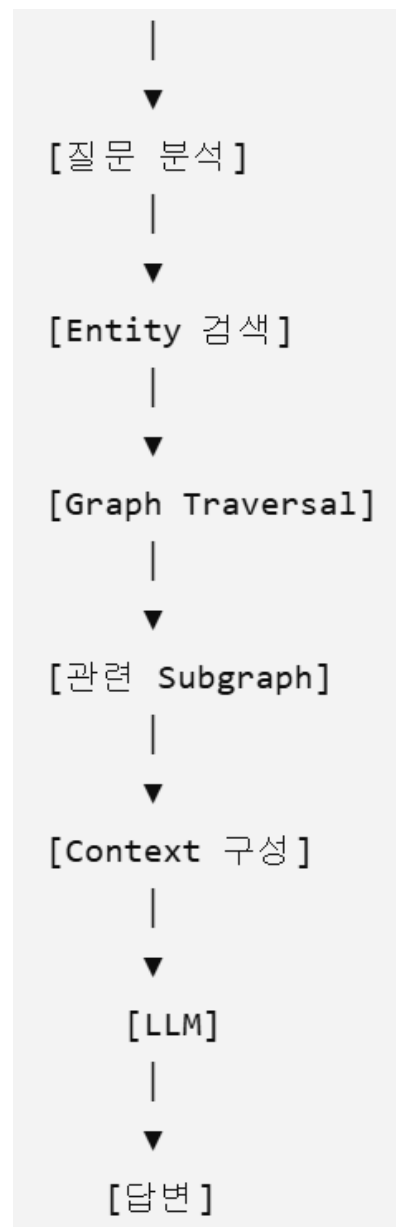
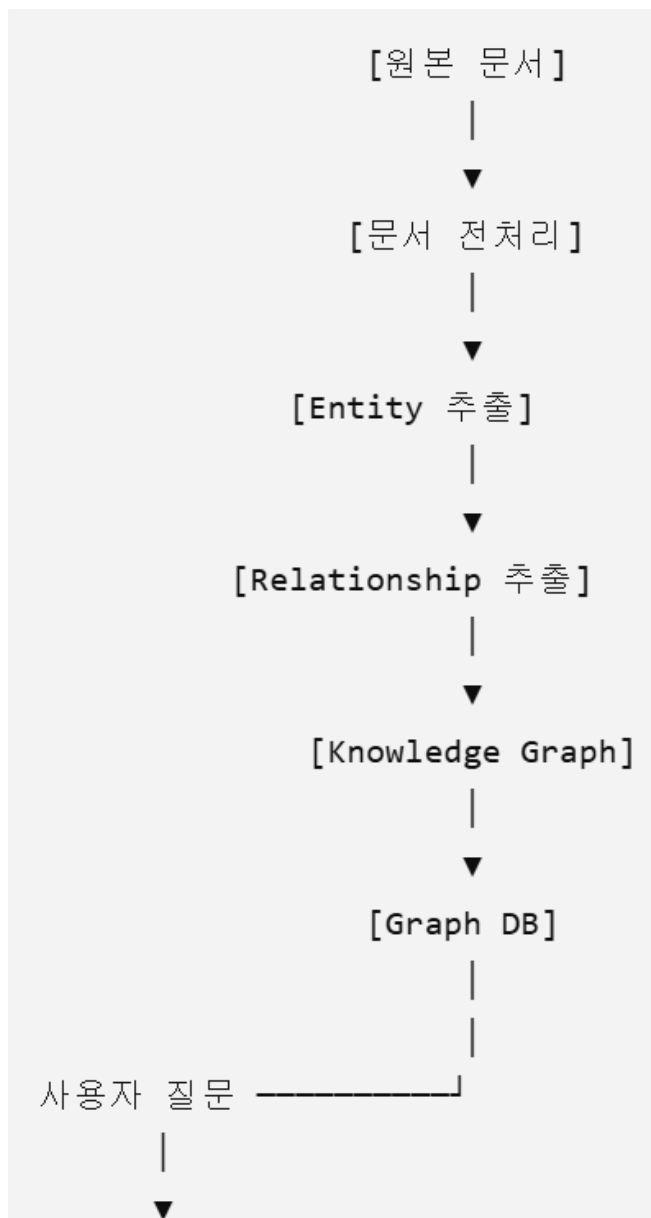
삼성전자
↓
HBM
↓
AI 가속기

④ Context

"검색된 관계 정보를 LLM에게 어떻게 전달할 것인가?"

Subgraph
↓
Context
↓
LLM
↓
Answer

32. Graph RAG 전체 구조 요약



Vector RAG와 Graph RAG 핵심 비교

질문 유형	적합한 검색
"HBM이 무엇인가?"	Vector RAG
"HBM의 특징은?"	Vector RAG
"삼성전자가 개발한 HBM은?"	Vector + Graph
"HBM과 관련된 기업은?"	Graph RAG
"삼성전자와 NVIDIA는 어떻게 연결되는가?"	Graph RAG
"A → B → C 관계를 설명하라"	Graph RAG
"여러 문서에 흩어진 관계를 연결하라"	Graph RAG

Vector RAG

→ "무엇이 비슷한가?"

Graph RAG

→ "무엇이 어떻게 연결되어 있는가?"



감사합니다